

COMP101

Tutorial-05

Pseudocode - Selection

Pseudocode: Selection

- A 'good' pseudocode design can be implemented in any language
- Pseudocode shows the logic of the problem – not the language
 - Use the language sub-set of pseudocode to design the program
- Don't use language-specific terms in pseudocode
 - e.g. Python does not use the 'then' statement – it uses 'elif'
 - we don't use elif in pseudocode - it is specific to Python
 - we use if – then – else as in the pseudocode language subset

Pseudocode: Selection problem statement

- A die (plural is 'dice') - a six-sided cube with each side of the cube showing an integer 1 to 6. There is one unique integer on each side
- A user 'throws' two die. If they throw a 'double six' they are told they can play the game again and the program ends
- If not a 'double six' they are told to pass the dice to another player and the program ends

Pseudocode: Draft solution

#no initialisation

1. input

1.1 input integer1

1.2 input integer2

2. process

2.1 calc total = integer1 + integer2

2.2 if total = 12 then

2.2.1 print 'Throw again'

2.3 else

2.3.1 print 'pass dice'

3. output

print game over

Notes:

No language-specific constructs used:

No 'elif' no delimiters;

No initialisation due to trivial design with limited i/o;

No implementation issues:

i.e. we will probably implement this with a random function to generate the dice numbers – but not in the pseudocode;

In actual implementation we could use a random range of 2 to 12.

If a 12 is randomly generated it would indicate we have two sixes – no other combination can produce a total = 12.

However – this implementation would still be true to the pseudocode

Pseudocode: No officially recognised standards programmers usually adopt a 'House Style'

```
1. input score
2. if score >=90
3. then
4. print A
5. if score >=80
6. then
7. print B
8. if score >=70
9. then
10. print C
11. if score >=60
12. then
13. print D
14. else
15. print Fail
```

This is just one example;
There may be no numbering or sub-numbering - the design instead relying on indentation etc to show structure and logic

There may be a slightly different wordset

The actual format of the pseudocode style is not critical.
It is critical that it shows the structure and the logic of what is to be programmed in any programming language

Nested if statements

- An if statement that has a statement inside it that is another if statement is 'nested'
- If we have excessive if statements, we should re-consider the logic of the program
- Ideally use sparingly to avoid complicated logic
 - Good indentation can help

Nested if's – keep it simple for the logic

```
IF condition-outer
  IF condition-inner
    statement-1
  ELSE
    statement-2
  END-IF
  statement-3
ELSE
  statement-4
END-IF
```

```
1. if condition-outer
  1.1 if condition-inner
    1.1.1 inner-statement-1
  1.2 else #1.1=false
    1.2.1 inner-statement-2
  1.3 end-if #end of 1.1 inner if
  1.4 outer-statement-1
2. else 1= false
  2.1 outer-statement-2
3. end-if #end of 1. outer if
```

Notes:

Sometimes numbering can help, sometimes it can confuse – too much detail?

Changing a name can help clarify logic?
e.g. 'statement-3' becomes 'outer-statement-1' and 'statement-4' becomes 'outer-statement-2' etc

Good use of indentation can show which statement belongs to which if:
Strengthened by the use of 'end-if'